

ETIS

Architecture & Detailed Design

Product architecture, trust boundaries, course model, evidence pipeline, review orchestration, security, lifecycle, and Azure production design

v0.18.0 manual edition. Software release candidate; publication follows deployment acceptance.

Audience	Engineering maintainers, security reviewers, architects, course-platform owners, advanced operators
Purpose	Explain why Studio is built the way it is and define the invariants future changes must preserve.
Reference implementation	COMP 330/474 Software Engineering • Fall 2026
Canonical service	https://simulator.etisframework.org
Version	v0.18.0 • October 2, 2026

CORE PRINCIPLE

AI may challenge, critique, synthesize, and ask for evidence. Engineers decide, defend, and own the consequences. Deterministic authorization, lifecycle state, and frozen repository evidence remain authoritative.

Contents

Executive Overview Architectural intent, responsibility boundaries, and key invariants

- 1 Product intent and explicit non-goals
- 2 Logical architecture
- 3 Identity and authorization model
- 4 Course, term, section, team, and membership model
- 5 GitHub identity and repository onboarding
- 6 Evidence acquisition, FACT/REVIEW, and frozen snapshots
- 7 Review orchestration and AI boundary
- 8 Student decision and learning-history model
- 9 Instructor/teaching-staff design
- 10 Persistence and schema lifecycle
- 11 Concurrency, idempotency, and fail-closed invariants
- 12 Azure production topology
- 13 Delivery, observability, recovery, and cost control
- 14 Security and privacy boundaries
- 15 Failure modes and defensive behavior
- 16 Extension points and change discipline

Appendix A Key invariants

Appendix B Source map

Appendix C Architecture decision test cases

Executive Overview

ETIS Engineering Studio is a decision-and-defense environment for engineering apprenticeship. Its core learning surface is the Engineering Review Room: students confront repository-derived evidence, uncertainty, tradeoffs, consequences, and professional responsibility. The architecture intentionally separates deterministic responsibilities from AI assistance. Authentication, course authorization, lifecycle state, repository trust, frozen evidence, and persistence are deterministic system responsibilities; AI is a bounded adviser.

ETIS Engineering Studio logical architecture

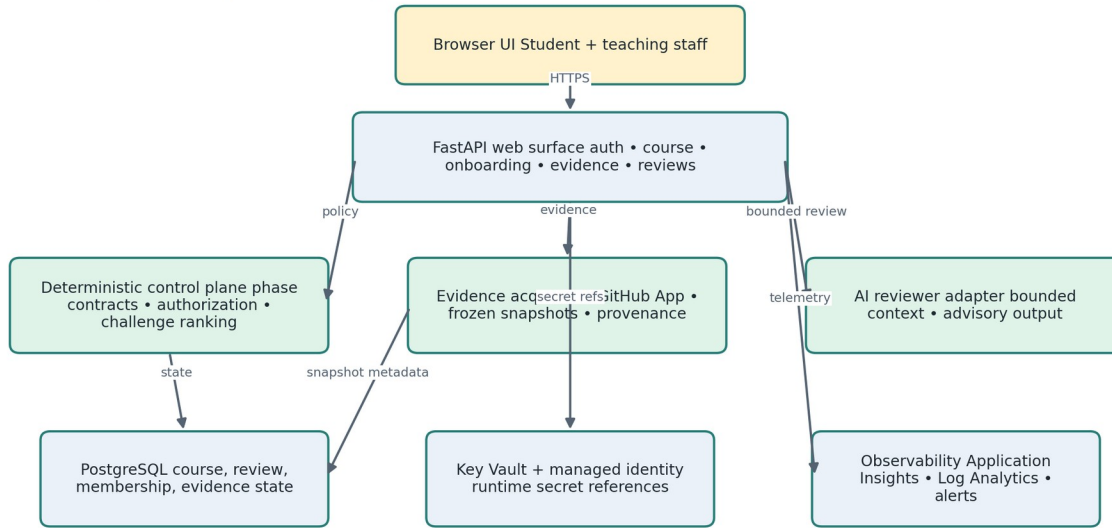


Figure 1. Logical architecture. Deterministic policy and frozen evidence remain authoritative; AI is advisory. Historical reference figure; current text governs.

ARCHITECTURAL RULE

The system may help a student reason, but it must never silently become the responsible engineer. Course requirements, authorization, snapshot evidence, and lifecycle state are deterministic system responsibilities.

1. Product intent and explicit non-goals

1.1 Intended outcomes

- Practice professional engineering judgment rather than answer retrieval.
- Force explicit evidence boundaries, uncertainty, tradeoffs, ownership, and consequences.
- Expose students to multiple professional lenses without treating AI reviewers as authorities.
- Preserve an inspectable engineering record of frozen evidence and persisted review turns.
- Allow disagreement and correction when REVIEW interpretations are wrong without mutating FACT evidence.

1.2 Explicit non-goals

- No autonomous final grading.
- No hidden assignment-scope expansion beyond instructor/course contracts.
- No fabrication of missing repository evidence.

- No student PATs for repository access.
- No generic application-global instructor role that bypasses term/section authority.
- No destructive archive semantics.
- No AI-generated mutation of the repository as a substitute for student engineering work.

2. Logical architecture

Component	Responsibility
Web surface	FastAPI serves bundled UI and API routes for authentication, course administration, onboarding, evidence, instructor views, and review sessions.
Course model	Machine-readable A1-A6 phase contracts define deterministic requirements, release context, and challenge anchors for the COMP 330/474 implementation.
Challenge control plane	Ranks evidence gaps, phase defenses, professional lenses, and engineering moves without delegating policy to the LLM.
Evidence services	GitHub App access, repository intelligence, frozen snapshot creation, evidence packages, strengths/findings, provenance.
AI services	Bounded reviewer adapter, semantic repository interpretation/coaching, cost/latency telemetry.
Persistence	SQLAlchemy models on PostgreSQL production / SQLite local; Alembic controls schema.
Operations	Azure health/readiness, monitoring, alerts, PITR, immutable ACR images, budget controls, operations CLI.

3. Identity and authorization model

The design deliberately separates human authentication, course authorization, GitHub identity, GitHub repository authority, and the active Studio session. Conflating any pair would make it possible for the wrong external identity or stale course role to mutate current state.

Identity, authorization, and repository trust boundaries

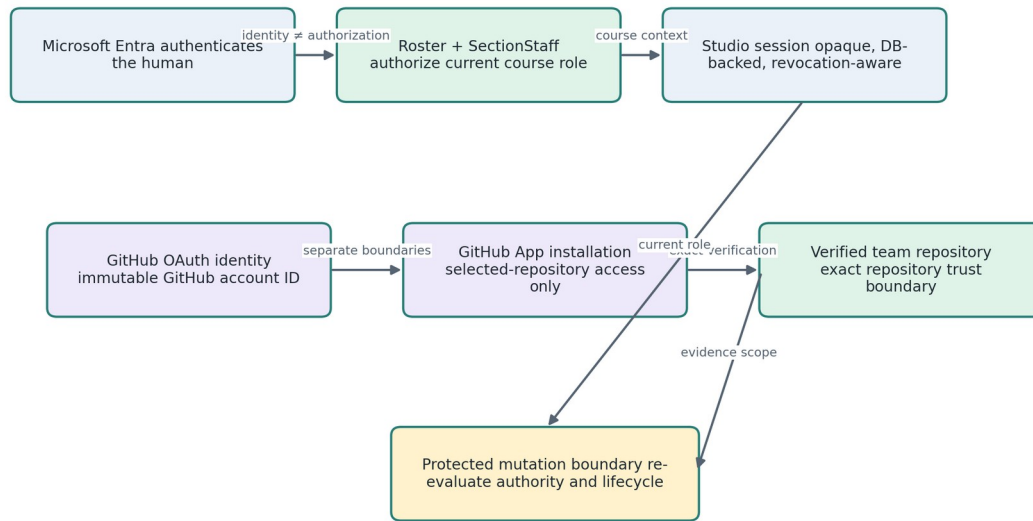


Figure 2. Identity, authorization, and repository trust are separate boundaries that converge only at protected action/evidence boundaries. Historical reference figure; current text governs.

Boundary	Design
Microsoft Entra	Authenticates the human. Production uses an explicit tenant and verified tenant-scoped object ID.
Roster / SectionStaff	Authorizes current course access and role. Lifecycle-valid database state is authoritative.
GitHub OAuth identity	Maps Studio user to immutable GitHub account ID + current login. OAuth state is bound to the initiating revocation-aware Studio session.
GitHub App installation	Authorizes read-only repository evidence access. Selected repositories only; exact repository token scope.
Studio session	Opaque, database-backed, revocation-aware, approximately 12-hour session in the accepted design. Current authorization is re-evaluated during parsing/mutation boundaries.

3.1 Staff roles

Role	Authority boundary
Course Owner	Term-scoped elevated administration; prepare/activate/archive terms, grant elevated staff authority, bounded repository reset, historical read.

Role	Authority boundary
Instructor	Assigned-section administration and historical read where permitted; bounded repository reset.
TA	Bounded active-semester review/read; no roster/lifecycle/elevated-role mutation; no archived authority.
Reviewer	Bounded active-semester review/read; no course administration; no archived authority.
Student	Current active section/team access and own permitted review activity.

4. Course, term, section, team, and membership model

Durable scope follows Course Template → Term → Section → Team → Student. Parallel sections share the COMP 330 lifecycle model but can differ in dates, release overrides, roster, teams, and staff.

Authoritative semester lifecycle

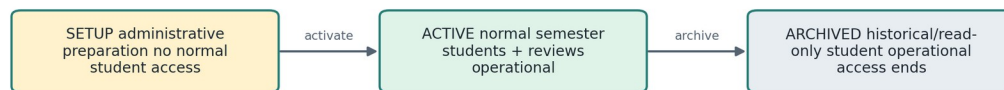


Figure 3. Semester lifecycle is forward-only: setup → active → archived. Archive preserves history but ends normal student operation. Historical reference figure; current text governs.

- Setup is administrative preparation.
- Active is current teaching state; authorization resolves from current enrollment, team membership, section staff, release state, and lifecycle.
- Archived is read-only history; student operational access ends; active reviews become archived_incomplete; historical evidence and conversations remain.
- Moving/dropping a student preserves earlier review history and membership events.

5. GitHub identity and repository onboarding

Repository onboarding state machine

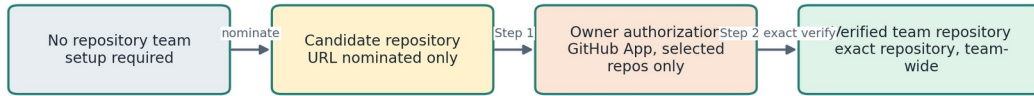


Figure 4. Team repository onboarding is a controlled state machine. A pasted URL is a nomination, not trusted evidence. Historical reference figure; current text governs.

5.1 Candidate versus verified repository

A typed URL is only a candidate. Candidate nomination cannot populate authoritative verified repository metadata and cannot be used as trusted evidence until exact GitHub verification succeeds.

5.2 Owner authority and installation/token invariants

- Repository ownership is resolved from namespace owner identity using immutable GitHub account ID where applicable.
- Personal repository: only the actual owner may authorize ETIS. Non-owning teammates wait; verification success becomes team-wide.
- Organization repository: GitHub's native owner/admin request/approval workflow determines who can grant access.
- The App is public so personal accounts can install it, but every installation must use selected repositories only.
- repository_selection=all fails closed.
- ETIS requests a short-lived installation token containing only the nominated repository and verifies the returned repository list exactly.
- PATs and long-lived OAuth repository credentials are not supported.
- Changing an individual GitHub identity does not silently replace a team's verified repository.

6. Evidence acquisition, FACT/REVIEW, and frozen snapshots

Frozen evidence and review continuity

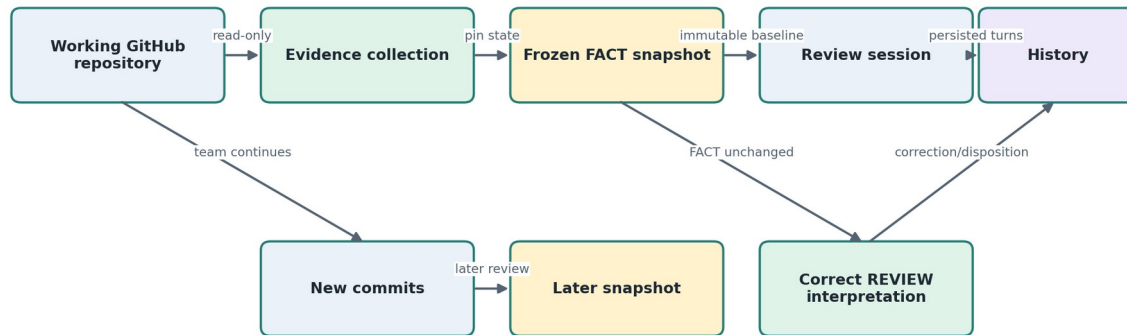


Figure 5. Evidence checkpoint lifecycle. New repository commits never mutate the FACT snapshot already bound to an active or completed review. Historical reference figure; current text governs.

Concept	Meaning
FACT	Immutable repository-derived baseline: paths, commit SHA/provenance, bounded artifact excerpts/metadata, workflow signals.
REVIEW	Interpretation of FACT evidence: findings, strengths, gaps, concerns. REVIEW may be corrected without changing FACT.
Snapshot	Pinned evidence package used by a review. Repository refresh never mutates evidence underneath an active review.
Correction/disposition	Validated correction or staff disposition prevents a later review on the same snapshot from rediscovering the same false interpretation while preserving original FACT.
Commit history	GitHub remains authoritative for raw commit history. ETIS may use commit/workflow history as evidence signals but does not replace GitHub as the source-control history browser.

6.1 Evidence acquisition scope

The collector reads bounded repository information authorized by the GitHub App: repository metadata, default-branch ref/tree/blobs, issues, pull requests/reviews, Actions runs, tags, commits, branches, and later-phase

release signals. The evidence layer converts those observations into phase-aware FACT and REVIEW structures before review.

7. Review orchestration and AI boundary

Engineering review control flow

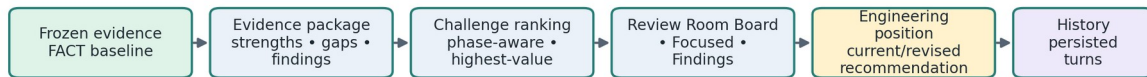


Figure 6. Review orchestration from frozen evidence through persisted learning history. AI operates inside the review step, not as the authority for evidence or course policy. Historical reference figure; current text governs.

7.1 Review types

Type	Purpose
Board Review	Normal developmental phase-gate review. The board ranks/selects the highest-value current challenge.
Focused Review	Student-selected artifact, decision, PR, architecture choice, risk, AI use, or work-in-progress concern.
Review Findings	Understand, challenge, resolve, accept/defer, or provide contrary evidence for one or a small related set of existing findings.

7.2 Reviewer lenses

Professional lenses include Chief Architect, Security Engineer, Verification Lead, Operations Lead, Human Impact Reviewer, and Red Team. They challenge from different risk perspectives; legitimate disagreement is preserved rather than flattened into one synthetic “correct” answer.

7.3 Bounded AI economics and context

- OpenAI receives compact, relevant evidence packages rather than entire repositories.
- Usage telemetry tracks model, input/cached-input/output tokens, latency, and estimated cost without duplicating complete prompts/responses into telemetry records.
- AI may fail or be unavailable; deterministic authorization and evidence policy still fail closed rather than inventing evidence.

Current evidence and analytical authority

The Review Room is the conversation home. Engineering Evidence is the saved source notebook. Same-snapshot and newer-saved-snapshot notices explain their relationship; browsing newer saved evidence never changes an active review's frozen baseline.

Inspect cited source opens the attached frozen artifact. Find supporting evidence searches the frozen snapshot for equivalent evidence; it is bounded and not exhaustive. Discuss helps you understand, Challenge tests an interpretation, and Help me resolve this coaches next work. An absent source must not be replaced by a guessed file link.

Only authoritative corrected or resolved findings are closed. Confirmed, evidence disputed, accepted risk and deferred findings remain open. Inspecting, searching, recommending and completing a review do not close a finding; work after the snapshot cannot close it on that earlier snapshot.

Production defaults remain legacy reasoning and legacy planning. Shadow validator/planner/selector/realizer evaluation remains internal and does not replace student-visible authority. This release introduces no PR5 authority transfer, database migration or course-model change.

The frozen snapshot is team evidence; reasoning and questions are student-session state. Phase-aware cumulative coaching must not reuse earlier-phase achievement as proof for a different current claim. Equivalent evidence can narrow or correct a false interpretation without treating filenames or scaffold as proof.

8. Student decision and learning-history model

- Current recommendation is optional and revisable; it tells the reviewer where the engineer is leaning.
- A student may remain “not ready to choose yet” while evidence or reasoning is incomplete.
- The recommendation posture changes the next engineering move/coaching challenge.
- Formal recommendation/discussion actions occur only when the student is prepared to defend a consequential position; not every review requires a final commitment.
- Review History preserves individual persisted turns against the frozen evidence used at that moment.
- Unsent drafts remain private.

DEVELOPMENTAL VS. FORMAL EVIDENCE

Studio dialogue is developmental. It may influence what the team changes, but it does not silently become formal phase-gate evidence. Formal review uses the instructor-designated repository evidence boundary/tag.

9. Instructor/teaching-staff design

- Instructor Command Center aggregates authorized active sections and makes section context explicit.
- Section selection persists across meaningful instructor views; browser Back/Forward restores meaningful context.
- Team Detail shows stable team identifier/context, membership, repository state, evidence signals, and active student reviews.
- Gold/amber means needs attention; green means no current attention signal. Attention is a coaching signal, not an automated grade.
- Teaching staff may inspect persisted review turns when authorized; generic read access never grants student mutation authority.
- Repository reset is bounded recovery: it clears the current verified binding while preserving frozen evidence/review history.
- TA/Reviewer roles remain intentionally read-oriented and cannot mutate elevated course administration or archived state.

10. Persistence and schema lifecycle

PostgreSQL is the production system of record. Local development may use SQLite. Alembic migrations are explicit release artifacts; production startup does not silently create or mutate schema.

- Production readiness compares current Alembic revision with application head.
- One Alembic head is required before release/deploy.
- Migrations execute in a Container Apps Job before application deployment.
- Duplicate immutable GitHub account IDs fail closed rather than merging identities.
- Archive retains historical evidence, review turns, findings/dispositions, membership events, and AI usage records according to lifecycle/retention policy.

11. Concurrency, idempotency, and fail-closed invariants

Invariant	Defensive behavior
Repository promotion race	After external GitHub verification, re-read/lock authoritative candidate state; if candidate changed, fail instead of promoting stale repository.
Team mutation authority	Re-evaluate membership/staff authority at the mutation boundary, not only request entry.
Session revocation	Archived/lost course authority revokes course-bound session access during parsing/authorization.
OAuth callback	Signed state alone is not authentication; callback requires the same initiating Studio session/user.
Side-effecting GET	Repository authorization navigation does not silently persist trust-state mutation; persisted transitions occur through explicit mutation behavior.
Malformed repository URL	Reject credentials, ports, query/fragment, extra path segments, invalid names, and non-HTTPS GitHub URLs.
All-repository GitHub installation	Fail closed; do not accept overly broad scope.
Schema/readiness mismatch	/ready is not ready; deployment acceptance fails rather than serving normal traffic.

12. Azure production topology

Accepted Azure production topology

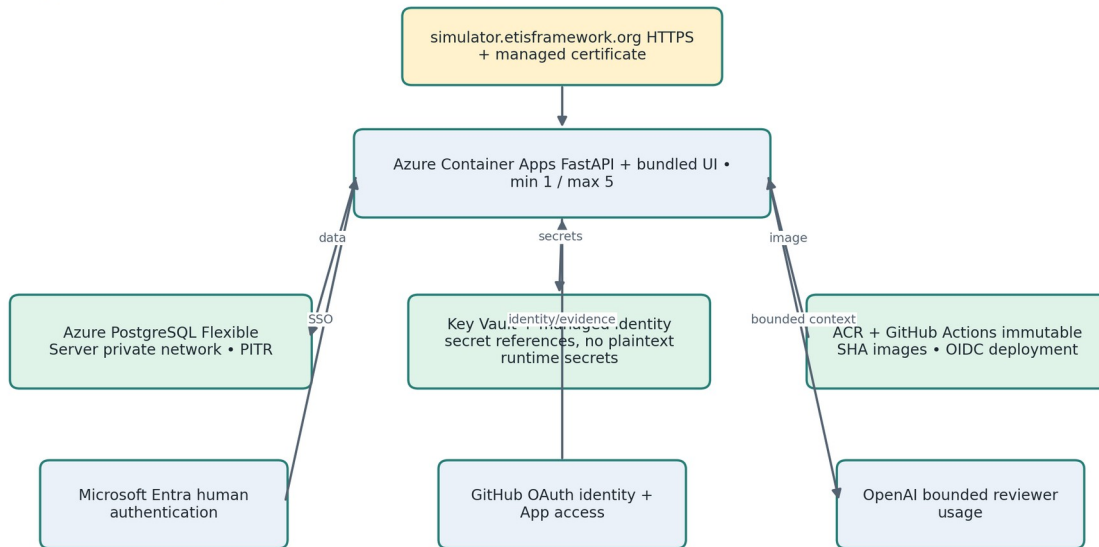


Figure 7. Azure production topology and external trust boundaries. Historical reference figure; current text governs.

Control	Accepted design
Container Apps	Public HTTPS FastAPI/bundled UI, Single revision mode, min 1/max 5 semester baseline.
PostgreSQL Flexible Server	PostgreSQL 16, private network, PITR/backup retention enabled and restore-tested.
Key Vault	Production secrets referenced at runtime; managed identity receives narrow RBAC.
ACR	Immutable application images tagged by Git commit SHA; managed identity pull.
Application Insights + Log Analytics	Workspace-based observability and operational telemetry.
Azure Monitor	Alerts for restarts, HTTP 5xx, PostgreSQL health/storage with operations action group.
Budget	\$100 monthly reference budget with 50/80/100% actual-cost notifications.

13. Delivery, observability, recovery, and cost control

Controlled production delivery path

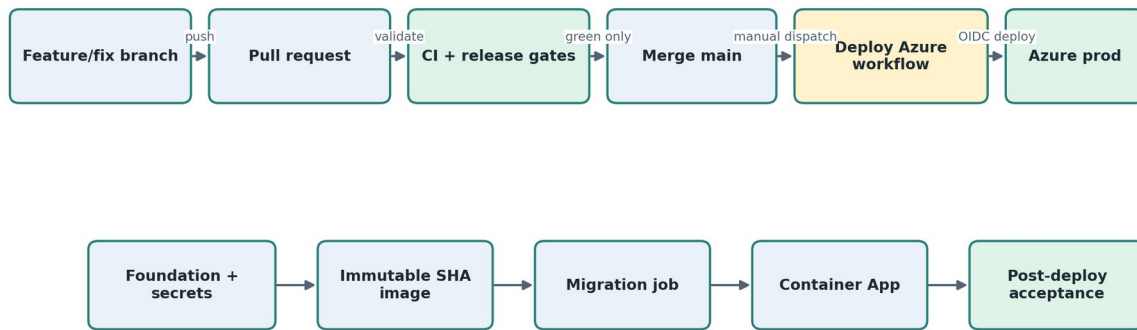


Figure 8. Production delivery is repository-controlled and revalidated before Azure authority is acquired. Historical reference figure; current text governs.

Production cost governance

Accepted monthly budget: \$100 • actual-cost notifications • advisory, not an automatic shutdown



Operator response: confirm scope → investigate usage → compare trend → act only if operationally justified

Figure 9. Cost governance is advisory operational control: thresholds notify operators but do not become an availability or grading mechanism. Historical reference figure; current text governs.

Operational control	Architectural purpose
/health	Liveness/service state. Should not claim full readiness when critical dependencies are unavailable.
/ready	Fail closed on database connectivity or migration mismatch.
PITR	Recover durable PostgreSQL state to a known point in time; validate restored schema/data before use.

Operational control	Architectural purpose
Rollback	Redeploy a prior immutable SHA-tagged image; Single revision mode avoids relying on a pile of simultaneously active revisions.
Drift	Compare live runtime with source/accepted baseline rather than assuming portal state matches IaC intent.
Budget/Cost	Stewardship telemetry. Cost APIs may lag/throttle; this is not a production-health gate.

14. Security and privacy boundaries

- No student Studio passwords; Entra is the human identity provider.
- No PATs; GitHub App short-lived exact-repository tokens only.
- No long-lived GitHub OAuth access-token retention for repository reading.
- No plaintext runtime secrets in Container App configuration.
- No complete prompts/responses in AI usage telemetry; telemetry is bounded metadata.
- Normal logs must not contain cookies, authorization codes, tokens, API keys, complete prompt bodies, complete model responses, or unnecessary student PII.
- Archive preserves historical engineering record; retention/deletion is a separate governance decision.
- Unsent student drafts remain private; staff conversation drill-down exposes persisted turns only.

15. Failure modes and defensive behavior

Failure	Expected behavior
GitHub App installed for all repositories	Verification fails with explicit selected-repository requirement.
Wrong GitHub account	Identity can be relinked; immutable account-ID conflict fails closed.
Organization approval pending	Student sees owner/admin authorization requirement rather than generic “instructor approval.”
Candidate changes mid-verification	Promotion returns conflict; authoritative team connection is not corrupted.
Archived term	Student operational access ends; stale staff authority cannot mutate unrelated semesters.
Database unavailable / schema behind	/ready is not ready; deployment acceptance fails.
AI unavailable	Do not fabricate evidence or silently claim semantic coaching occurred.
Browser GitHub authorization tab	Setup URL returns to ETIS; exact repository verification remains Step 2.

Failure	Expected behavior
Cost API throttled	Report advisory WARN; do not mark application unhealthy or repeatedly hammer API.

16. Extension points and change discipline

- Deeper A3-A6 conversational depth and specialized engineering stress scenarios.
- Advanced instructor analytics, annotations, replay/archive UI, and research exports.
- Push notifications and finer-grained cost/rate controls.
- Explicit accessibility/performance instrumentation as separate controlled workstreams.
- Institution-neutral course templates beyond the COMP 330/474 six-gate implementation.

CHANGE DISCIPLINE

Do not reopen accepted trust boundaries for speculative polish. Future changes should be driven by an observed defect, course need, security requirement, or intentionally scheduled enhancement and must preserve Appendix A invariants.

Version and UX invariants

The v0.18.0 source version is defined in apps/api/app/version.py. FastAPI/OpenAPI and /health use that value. Source version, merged Git SHA, immutable image/revision, deployment success and production acceptance are separate evidence. The August GO records and v0.17 analytical history keep their original dates and versions.

Button-triggered instructions are recorded as review actions, not as words the student wrote. Unsent drafts are preserved and remain private. Optional review context and other concerns stay secondary to the current concern and conversation.

In Help, artifact, challenge and exit dialogs, Tab and Shift+Tab stay within the dialog, Escape dismisses through the existing close action, and focus returns to a usable opener. Escape does not finish a review. Closing a pending challenge does not cancel its submitted request.

Release validation includes focused tests, existing offline journey and keyboard war games, and complete pytest. A1–A6 require weak/average/strong repository and learner cases with missing, equivalent, ambiguous, contradictory, stale and post-snapshot evidence, valid challenges, false positives/negatives, overclaiming, grading and UX failures. Offline tests do not prove live semantic reviewer quality.

Appendix A. Key invariants

- ☐ Authentication and course authorization remain separate.
- ☐ Archived semesters never grant current mutation authority.
- ☐ A candidate repository is never trusted evidence.
- ☐ Only the repository owner/authorized organization can grant GitHub App access.
- ☐ All-repositories GitHub App scope is rejected.
- ☐ Exact-repository installation token scope is verified.
- ☐ Frozen FACT evidence is immutable.
- ☐ Correctable REVIEW interpretation is not FACT evidence.
- ☐ Read authority does not imply student mutation authority.

- ☐ Unsent drafts remain private.
- ☐ Schema changes go through Alembic.
- ☐ Production secrets remain in Key Vault / protected deployment secrets.
- ☐ Health/readiness fail closed on critical dependencies.
- ☐ Cost telemetry does not silently become an availability or grading gate.
- ☐ Formal phase-gate evidence boundary remains distinct from developmental Studio dialogue.

Appendix B. Source map

Source	Primary design authority
docs/ARCHITECTURE.md	Product/logical architecture
docs/SECURITY_AND_PRIVACY.md	Privacy, logging, GitHub/AI boundaries
docs/architecture/ IDENTITY_COURSE_ADMINISTRATION.md	Identity, roles, lifecycle, onboarding
docs/architecture/ REPOSITORY_INTELLIGENCE_AND_REVIEW_ORCHESTRATION.md	Evidence/repository/review orchestration
docs/architecture/ ENGINEERING_EVIDENCE_AND_REVIEW_CONTINUITY. md	Evidence workspace and review continuity
apps/api/app/services/auth.py	Session, Entra, OAuth state and identity binding
apps/api/app/services/github_app.py	GitHub owner resolution and exact-repository tokens
apps/api/app/routers/onboarding.py	Repository onboarding state transitions
apps/api/app/routers/reviews.py	Review mutations and authority
infra/azure/*.bicep	Production IaC layers
.github/workflows/deploy-azure.yml	Release validation/deployment order
scripts/azure/	Operator-facing production inspection, drift, smoke, acceptance, and guarded scale override

Appendix C. Architecture decision test cases

Scenario	Required architectural result
Student signs in with valid Loyola account but is not rostered	Authenticate human; deny course operation.

Scenario	Required architectural result
Student pastes a GitHub URL	Store candidate only; do not trust evidence.
App installation has All repositories	Fail exact verification.
Team changes candidate while verification is in flight	Do not promote stale candidate.
Repository receives new commits during active review	Active review remains bound to frozen snapshot.
Reviewer interpretation is proven wrong	Correct/disposition REVIEW without mutating FACT snapshot.
TA can read a review	TA still cannot perform student review mutation or elevated course administration.
Term archived while review active	End normal student access; preserve review as archived_incomplete.
Database migration behind application head	/ready fails; deployment acceptance blocks normal service.
OpenAI unavailable	Preserve deterministic controls; do not fabricate a reviewer result.
Cost API returns 429	Report advisory WARN; production health remains determined by required checks.